

Assingment 1
Aidos Jambulatov
SE-2521

1. Asymptotic Bounds

Algorithm, Best case, Average case, Worst case, Reason

Insertion Sort,

$\Theta(n)$ Best: already sorted, inner loop stops after 1 comparison each time.

$\Theta(n^2)$ Average: $\sim n^2/4$ or $n^2/2$ swaps for random/reverse-sorted input.

$\Theta(n^2)$ Worst: $\sim n^2/4$ or $n^2/2$ swaps for random/reverse-sorted input.

MergeSort,

$\Theta(n \log n)$ Best

$\Theta(n \log n)$ Average

$\Theta(n \log n)$, Worst

Split is always exactly in half regardless of input, and merge is always linear, so all three cases coincide.

QuickSort (random pivot, 3-way, smaller-side-first)

$\Theta(n \log n)$ Best: a random pivot splits the array into two roughly-comparable parts w.h.p.

$\Theta(n \log n)$ Average: a random pivot splits the array into two roughly-comparable parts w.h.p.

$O(n \log n)$ ($\Omega(n^2)$) Worst case for a classic QuickSort is $\Theta(n^2)$ on adversarial pivots (e.g. always picking the min/max, as happens with a fixed pivot on sorted input) — random pivot selection makes that case astronomically unlikely rather than impossible, so we state $O(n \log n)$ with overwhelming probability instead of a tight worst-case Θ .

QuickSelect (random pivot, one-side recursion)

$\Theta(n)$ Best: each partition step is expected to discard a constant fraction of the array, giving $T(n) = T(n/2) + O(n) \Rightarrow \Theta(n)$ (see recurrence below).

$\Theta(n)$ Average: each partition step is expected to discard a constant fraction of the array, giving $T(n) = T(n/2) + O(n) \Rightarrow \Theta(n)$ (see recurrence below).

$O(n^2)$, $\Theta(n)$ Worst case (if the random pivot happened to repeatedly be the min/max) is $\Theta(n^2)$, again astronomically unlikely with random pivots — the Median-of-Medians bonus task removes this worst case entirely by guaranteeing a good pivot deterministically.

2. Recurrences

2.1 MergeSort

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

$$a = 2, b = 2, f(n) = \Theta(n)$$

$$n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n, \text{ which matches } f(n) = \Theta(n)$$

$$\text{Master Theorem Case 2 } (f(n) = \Theta(n^{(\log_b a)})) \Rightarrow T(n) = \Theta(n \log n)$$

This holds for best, average and worst case alike, because MergeSort's split point and merge cost never depend on the input's order — only on n .

2.2 QuickSort

Assuming a balanced split (pivot always near the median):

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

$$a = 2, b = 2, f(n) = \Theta(n) \text{ (the 3-way partition pass is linear)}$$

$$\text{Master Theorem Case 2} \Rightarrow T(n) = \Theta(n \log n) \text{ under the balanced-split assumption.}$$

Why a random pivot gives $O(n \log n)$ on average even without a "true" balanced split: A single random pivot will not always land exactly at the median, but on average it lands somewhere in the "middle half" of the sorted order (between the 25th and 75th percentile) with probability $1/2$. Even a sequence of only moderately unbalanced splits (say, $1/4$ – $3/4$ in the worst realistic case) still yields recursion depth $O(\log n)$ and total work $O(n \log n)$, because the sum of work per recursion level stays $\Theta(n)$ regardless of how lopsided any single split is, as long as splits don't degenerate to $O(1)/O(n)$ every single time. Since the pivot is re-randomized independently at every call, the probability of getting a long run of consistently bad ($O(1)/O(n)$) splits shrinks exponentially, so the expected running time over all random choices is $\Theta(n \log n)$.

2.3 QuickSelect

Assuming a balanced split (pivot near the median), QuickSelect recurses into only one side:

$$T(n) = 1 \cdot T(n/2) + \Theta(n)$$

$$a = 1, b = 2, f(n) = \Theta(n)$$

$$n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1, \text{ and } f(n) = \Theta(n) \text{ grows polynomially faster than } n^0$$

Master Theorem Case 3 ($f(n) = \Omega(n^{\log_b a + \epsilon})$) and the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds for $a \cdot f(n/b) = f(n)/2 \leq c \cdot f(n) \Rightarrow T(n) = \Theta(f(n)) = \Theta(n)$

This is a different Master Theorem case from MergeSort/QuickSort (Case 3 instead of Case 2), because only one subproblem is solved instead of two — the work done partitioning at the top level ($\Theta(n)$) already dominates all the (shrinking, geometric) work done at every deeper level, so the total is $\Theta(n)$ rather than $\Theta(n \log n)$.

4. Θ Check

MergeSort, all three input types: the ratio comparisons / ($n \log_2 n$) is essentially flat from $n = 1\,000$ to $n = 1\,000\,000$, sitting around $c \approx 0.9\text{--}1.0$ (random/duplicates) — this is a clean confirmation of the $\Theta(n \log n)$ bound, since $c_1 \approx 0.85$ and $c_2 \approx 1.05$ already sandwich the curve for $n_0 = 1\,000$.

QuickSort, random/sorted input: the ratio also flattens around $c \approx 1.25\text{--}1.4$, confirming $\Theta(n \log n)$ on non-adversarial inputs, with $c_1 \approx 1.2$, $c_2 \approx 1.45$, $n_0 = 1\,000$.

QuickSort, duplicates: the ratio is much lower ($\approx 0.2\text{--}0.3$) and decreasing — the 3-way partition discards the " $=$ pivot" band entirely once it is found, so with only 10 distinct values the effective problem shrinks much faster than $n \log n$ predicts; this is expected and is not a violation of the upper bound, just a looser one for this specific input family.

QuickSelect: the ratio comparisons / n stays roughly bounded between 2 and 4.5 across all n and input types (it does not grow with n), which matches the $\Theta(n)$ prediction; it fluctuates run-to-run more than the sort ratios because a single random pivot choice near the top of the recursion has an outsized effect on a run that is much shorter overall than a full sort.

Overall: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ holds from roughly $n_0 = 1\,000$ onward for every algorithm/input combination measured, i.e. the constants stabilize almost immediately rather than only at very large n — a sign that $\Theta(n \log n)$ / $\Theta(n)$ are the right (tight) bounds, not just valid upper bounds.

5. Discussion

The measurements match the theory well. MergeSort and QuickSort both show a near-flat comparisons/($n \log n$) ratio across three orders of magnitude of n , which is the textbook signature of $\Theta(n \log n)$ — the ratio plot is a much more reliable way to "see" a Θ bound than the raw time-vs- n plot, because wall-clock time is also affected by constant factors that have nothing to do with the algorithm's asymptotic complexity (JIT warm-up, CPU cache behavior, allocation). Indeed, `time_vs_n.png` shows MergeSort and QuickSort tracking each other closely for random input but diverging sharply for sorted input, where QuickSort has to do actual partitioning work (comparisons close to

random-case levels) while MergeSort's early-exit merge check ($a[\text{mid}] \leq a[\text{mid}+1]$) makes each merge $O(n)$ with almost no data movement, so MergeSort ends up faster in wall-clock time on sorted input despite doing the "same" $\Theta(n \log n)$ asymptotic work — this shows the difference between asymptotic comparisons count and actual measured time, which also depends on how comparison-heavy vs. memory-movement-heavy each pass is.

The recursion-depth plot confirms the safety requirement: QuickSort's max depth on a sorted array of 1 000 000 elements is only $\sim 12\text{--}13$, comfortably under $2 \cdot \log_2(n) \approx 40$, thanks to the smaller-side-first + while-loop strategy — with a naive "always recurse left, then right" QuickSort this same input would recurse to depth n and StackOverflow. MergeSort's depth grows slightly faster (~ 18 at $n = 1\,000\,000$) because it always fully halves regardless of input, but is bounded by $\log_2(n/\text{CUTOFF}) \approx 16$, plus a small constant — matching the measured value closely.

Differences between the predicted $\Theta(n \log n)$ shape and the raw millisecond numbers come from several practical sources. JVM warm-up: the JIT compiler only optimizes hot code paths after enough invocations, so the very first calls in a run are slower than later ones — this is why the benchmark takes the median of 5 repeated runs per case rather than a single measurement. Garbage collector: MergeSort's helper buffer and QuickSort's implicit recursion frames both create allocation pressure; a GC pause landing inside one particular run can make that run an outlier, which the median-of-5 also filters out. CPU cache: sequential array access (as in the merge step, or in partitioning a contiguous range) is cache-friendly, while QuickSort's swaps during 3-way partitioning cause somewhat more scattered memory writes, which shows up as extra wall-clock time not reflected in the comparison count. Cutoff size (15): switching to Insertion Sort below this threshold reduces both the constant factor and the recursion depth for the smallest subarrays, trading a $\Theta(n^2)$ algorithm on tiny inputs (where the n^2 term is negligible) for lower overhead than a full recursive call — this is visible in the ratio plot as MergeSort's ratio being very slightly lower than the pure $\Theta(n \log n)$ prediction would suggest for the smallest n , since a chunk of the work below the cutoff is being done by an $O(n)$ or $O(n^2)$ -but-tiny insertion sort instead of by more recursive $\Theta(n \log n)$ merge calls.